

Welcome to VLSITHON 3.0

Instructions:

Please read carefully before you proceed

- Each Testcase carries 20 points
- Please use the provided answer sheet to answer Testcase#1, Testcase#4 and Testcase#5
- No extra sheet will be provided
- Testcase#2 must be compiled on the server
- Testcase#3 will require EDA tool access
- For EDA tools usage: the time (minutes) you spend using the tool will be deducted from your total points from all testcases, using less keeps your points, using more deducts your points
- Do not remove or modify any generated files; doing so will disqualify the team
- Self-interpretation of this question paper is part of the test
- This is a test of the participants own mind, use of internet, AI or any other external material is strictly prohibited

Testcase#1	Power Grid Sizing from Synthesis Power Report
Testcase#2	Matrix Multiplication Using MAC
Testcase#3	Setup Timing
Testcase#4	Logical Reasoning
Testcase#5	One Logic Function, Two Design Goals

Testcase#1: Power Grid Sizing from Synthesis Power Report

A synthesized digital design implemented in **GPDK045** has the following power report:

Instance: /darksocv
Power Unit: W

Category	Leakage	Internal	Switching	Total	Row%
register	6.40858e-04	4.68824e-01	1.66362e-02	4.86101e-01	65.51%
logic	8.86268e-05	1.08195e-01	1.47647e-01	2.55930e-01	34.49%
Subtotal	7.29484e-04	5.77019e-01	1.64283e-01	7.42032e-01	100.00%

The power grid will be implemented using **M5 power stripes**.

Given

Parameter	Value
Supply voltage (VDD)	1.1 V
Power-grid safety margin	20%
M5 stripe width	4 μm
Maximum continuous current per unit width on M5	8 mA/ μm

Assume each **VDD stripe must collectively support the full supply current**, and the same number of **VSS stripes** will be used.

Task

Using the synthesis power report and the given technology information:

1. Calculate the **total current drawn by the design**
2. Apply the **20% safety margin** to determine the required power-grid current capacity
3. Calculate the maximum current that can be carried by one M5 stripe
4. Determine the **minimum number of VDD stripes required**.

Show all calculations, units, and rounding decisions.

Testcase#2: Matrix Multiplication Using MAC

You are given a text file containing two square integer matrices, A and B.

The first value in the file specifies the matrix dimension, N. This is followed by the $N \times N$ elements of matrix A and then the $N \times N$ elements of matrix B.

Write a C program that reads the input file, performs matrix multiplication using Multiply–Accumulate (MAC) operations, and writes the resulting matrix C to an output file.

Each element of the output matrix is calculated as:

$$C[i][j] = \sum_{k=0}^{N-1} A[i][k] \times B[k][j]$$

For example:

$$C[0][0] = 1 \times 9 + 2 \times 6 + 3 \times 3 = 9 + 12 + 9 = 30$$

Example Input	Expected Output
3 1 2 3 4 5 6 7 8 9 9 8 7 6 5 4 3 2 1	30 24 18 84 69 54 138 114 90

Requirements

- The solution must be written in C.
- The matrix dimension N must be read from the input file.
- Do not hard-code the matrix size.
- Read both matrices from the supplied input file.
- Perform the multiplication using MAC operations.
- Preserve the order and dimensions of the resulting matrix.
- Write only the resulting $N \times N$ matrix to the output file.
- Include appropriate handling for invalid file access or input data.
- You must modify the mac.c file to write your code, use vim/emacs/gvim
- You must use the input.txt file as the input and write out output.txt
- Type “make testcase2” and enter on your linux terminal once you are done writing mac.c to test

Testcase#3: Setup Timing

You are given a routed database with a few remaining setup violations:

Path Name	Startpoint	Endpoint	Slack (ns)
pathA	core0/XIDATA_reg[13]/Q	core0/REGS_reg[3][18]/D	-0.419
pathB	core0/XIDATA_reg[10]/Q	core0/REGS_reg[11][24]/SE	-0.018
pathC	core0/FLUSH_reg[0]/Q	core0/XIDATA_reg[16]/D	-0.569

Fix the timing paths following the restrictions at the end of this page.

To open the database, type the following on your terminal and press enter:

```
make testcase3
```

Once the database is open, use the following commands on your eda terminal

Action	Command (case sensitive)
To load a path	pathX_load
To check slack and time spent	pathX_check
To submit and see time spent	pathX_submit

Replace X with A, B, C

Note:

- You must “Commit What-if” once fixed
- The number of minutes you spend between load and submit will be used to determine your point for this testcase

pathA

Fix the setup violation with following restrictions:

- Only allowed to change cells in data path
- No modification to the clock path

pathB

Fix the setup violation with following restrictions:

- Only allowed to add or remove buffers in data path
- No modification to the clock path

pathC

Fix the setup violation with following restrictions:

- No modification to the data path

pathA: VIOLATED Setup Check with Pin core0/REGS_reg[3][18]/CK

Endpoint: core0/REGS_reg[3][18]/D (v) checked with leading edge of **CLK0**
 Beginpoint: core0/XIDATA_reg[13]/Q (^) triggered by leading edge of **CLK0**

Path Groups: {CLK}

Analysis View: func_slow_125_lv62

Other End Arrival Time 0.804
 - Setup 0.302
 + Phase Shift 4.800
 = Required Time 5.302
 - Arrival Time 5.720
 = Slack Time -0.419

Clock Rise Edge 0.000
 + Clock Network Latency (Prop) 0.725
 = Beginpoint Arrival Time 0.725

Timing Point	Cell	Arc	Fanout	Load	Slew	Delay	Incr Delay	Arrival Time
core0/XIDATA_reg[13]/CK		CK ^	1	0.026	0.088			0.725
core0/XIDATA_reg[13]/Q ->	DFFX4	CK ^ -> Q ^	10	0.187	0.537	0.921	0.047	1.646
core0/FE_OFC415_XIDATA_13/Y	CLKINVX2	A ^ -> Y v	3	0.015	0.150	0.258	0.002	1.904
core0/g88896/Y	NAND2X2	B v -> Y ^	5	0.117	0.663	0.625	0.108	2.529
core0/g88588/Y	OR2X1	B ^ -> Y ^	2	0.114	1.221	1.429	0.254	3.958
core0/g137298/Y	A0I21X2_HS	B0 ^ -> Y v	1	0.008	0.212	0.204	0.000	4.162
core0/g137287/Y	INVX2_HS	A v -> Y ^	1	0.015	0.111	0.139	0.000	4.301
core0/g137100/Y	NAND2X4_HS	A ^ -> Y v	1	0.049	0.148	0.174	0.011	4.475
core0/FE_OPC5038_n_1258/Y	INVX8_HS	A v -> Y ^	9	0.110	0.141	0.171	0.000	4.646
core0/FE_RC_2035_0/Y	CLKAND2X2	B ^ -> Y ^	1	0.014	0.094	0.225	0.000	4.871
core0/FE_RC_2381_0/Y	NAND3X4_HS	B ^ -> Y v	1	0.040	0.202	0.213	0.012	5.083
core0/FE_RC_2033_0/Y	NAND2X4_HS	B v -> Y ^	2	0.119	0.331	0.330	0.048	5.414
core0/g136646_0_dup/Y	NAND2X4_HS	A ^ -> Y v	15	0.083	0.269	0.305	0.001	5.718
core0/REGS_reg[3][18]/D ->	SDDFX1_HS	D v		0.083	0.269	0.002	0.000	5.720

pathB: VIOLATED Setup Check with Pin core0/REGS_reg[11][24]/CK

Endpoint: core0/REGS_reg[11][24]/SE (v) checked with leading edge of **CLK0**
 Beginpoint: core0/XIDATA_reg[10]/Q (^) triggered by leading edge of **CLK0**

Path Groups: {CLK}

Analysis View: func_slow_125_lv62

Other End Arrival Time 0.806
 - Setup 0.439
 + Phase Shift 4.800
 = Required Time 5.167
 - Arrival Time 5.185
 = Slack Time -0.018

Clock Rise Edge 0.000
 + Clock Network Latency (Prop) 0.539
 = Beginpoint Arrival Time 0.539

Timing Point	Cell	Arc	Fanout	Load	Slew	Delay	Incr Delay	Arrival Time
core0/XIDATA_reg[10]/CK		CK ^	34	0.164	0.146			0.539
core0/XIDATA_reg[10]/Q ->	DFFX4_HS	CK ^ -> Q ^	6	0.160	0.340	0.686	0.010	1.226
core0/g138523/Y	NAND2X4_HS	B ^ -> Y v	7	0.072	0.240	0.310	0.001	1.536
core0/g138227/Y	OR2X1_HS	A v -> Y v	5	0.032	0.194	0.429	0.000	1.965
core0/g137999/Y	NOR2X1	B v -> Y ^	1	0.008	0.229	0.243	0.000	2.208
core0/FE_OFC216_n_559/Y	BUFX4	A ^ -> Y ^	12	0.168	0.481	0.576	0.042	2.784
core0/FE_OFC217_n_559/Y	CLKBUFX2	A ^ -> Y ^	11	0.202	1.110	1.130	0.061	3.913
core0/FE_OFC218_n_559/Y	INVX4	A ^ -> Y v	29	0.527	0.948	1.217	0.000	5.131
core0/REGS_reg[11][24]/SE ->	SDDFX1_HS	SE v		0.527	0.959	0.055	0.000	5.185

```

pathC: VIOLATED Setup Check with Pin core0/XIDATA_reg[16]/CK
Endpoint: core0/XIDATA_reg[16]/D (v) checked with leading edge of 'CLK'
Beginpoint: core0/FLUSH_reg[0]/Q (v) triggered by leading edge of 'CLK'
Path Groups: {CLK}
Analysis View: func_slow_125_lv62
Other End Arrival Time 0.374
- Setup 0.170
+ Phase Shift 4.800
= Required Time 5.004
- Arrival Time 5.574
= Slack Time -0.569

```

```

Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
Timing Path:

```

Timing Point	Cell	Arc	Fanout	Load	Slew	Delay	Incr Delay	Arrival Time
CLK		CLK ^	4	0.120	0.003			0.000
core0/CTS_ccl_a_buf_00037/Y	CLKBUF8_HS	A ^ -> Y ^	8	0.195	0.167	0.236	0.000	0.236
core0/CTS_cdb_buf_00059/Y	CLKBUF2_LP	A ^ -> Y ^	1	0.008	0.065	0.187	0.000	0.422
core0/CTS_ccl_a_buf_00004/Y	CLKBUF8_HS	A ^ -> Y ^	34	0.150	0.134	0.237	0.003	0.660
core0/FLUSH_reg[0]/Q ->	DFFX1	CK ^ -> Q v	3	0.024	0.167	0.634	0.002	1.293
core0/g89113/Y	NOR2X4_HS	A v -> Y ^	9	0.043	0.214	0.241	0.000	1.534
core0/FE_0FC873_n_64/Y	CLKINVX2	A ^ -> Y v	3	0.016	0.091	0.147	0.000	1.681
core0/g88647/Y	NOR2X1	A v -> Y ^	1	0.004	0.138	0.162	0.000	1.843
core0/FE_0FC5154_XXDREQ/Y	BUF4_HS	A ^ -> Y v	3	0.155	0.332	0.481	0.107	2.324
core0/g145737/Y	NAND2X2	A ^ -> Y v	1	0.014	0.130	0.227	0.000	2.551
core0/FE_0FC5150_n_15456/Y	BUF16_HS	A v -> Y v	48	0.412	0.180	0.280	0.000	2.831
core0/g145453/Y	NAND2X2	B v -> Y ^	3	0.019	0.176	0.284	0.000	3.114
core0/g142234/Y	CLKINVX4	A ^ -> Y v	6	0.045	0.101	0.156	0.000	3.270
core0/g142236/Y	NAND2X1	B v -> Y ^	1	0.011	0.271	0.155	0.000	3.425
core0/g137820/Y	INVX4	A ^ -> Y v	23	0.146	0.258	0.324	0.000	3.749
core0/g137562/Y	A0I22X1_HS	B1 v -> Y ^	2	0.076	1.264	1.464	0.465	5.213
core0/g140382/Y	NAND2X1	B ^ -> Y v	1	0.005	0.264	0.360	0.002	5.573
core0/XIDATA_reg[16]/D ->	DFFX4_HS	D v		0.005	0.264	0.000	0.000	5.574

```

Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
Other End Path:

```

Timing Point	Cell	Arc	Fanout	Load	Slew	Delay	Incr Delay	Arrival Time
CLK		CLK ^	4	0.117	0.003			0.000
core0/CTS_ccl_a_buf_00034/Y	CLKBUF8_HS	A ^ -> Y ^	3	0.061	0.075	0.163	0.000	0.163
core0/CTS_ccl_a_buf_00006/Y	BUF16	A ^ -> Y ^	11	0.081	0.087	0.210	0.000	0.373
core0/XIDATA_reg[16]/CK	DFFX4_HS	CK ^		0.081	0.087	0.001	0.000	0.374

Important Notes:

- To avoid losing too many points analyze the timing paths from this paper and only open EDA tool when you have a plan to fix the path
- Each paths time count starts when pathX_load is used
- Do not load mulitple paths to avoid losing X2/X3 points
- Using pathX_load will isolate the path you need to fix
- Use pathX_check from time to time to check how many minutes you have spent on the path
- Use pathX_submit when you have a fix to stop the time count
- Use pathX_submit to give up fixing the path and stop the time count to stop losing points
- Using pathX_submit will close the EDA tool. You can use “make testcase3” again to open the database and start woking on anohter path by invoking “pathX_load”
- Always use pathX_check before pathX_submit to confirm you have achieved positive slack

Testcase#4: Logical Reasoning

A fabricated chip works at 100 MHz but fails intermittently at 400 MHz.

Failures increase at high temperature and low supply voltage.

Increasing clock period fixes the problem.

Adding 100 mV supply also fixes it.

Four possible suspects:

- Hold violation
- Setup violation
- Antenna violation
- DRC spacing violation

Determine the most likely cause and explain why the observations support or contradict each suspect.

Testcase#5: One Logic Function, Two Design Goals

A standard-cell library team is developing two kits for the same technology node:

- **High-Performance Kit (HPK)**
- **Power Management Kit (PMK)**

Both kits use the same cell names, logic functions, and physical footprints, so implementation engineers can swap one version for the other without changing the design structure.

An engineer proposes the following shortcut:

"We only need to design each cell once. For the HPK version, we use LVT devices; for the PMK version, we just replace them with HVT devices. No other design changes are necessary."

Question

Apart from threshold-voltage (V_t) flavour, identify **two** transistor-level or cell-level design decisions that should reasonably differ between the HPK and PMK versions of the same logic gate.

For each decision:

1. Explain how the HPK and PMK implementations should differ.
2. State the benefit each kit gains from its choice.
3. State the trade-off or penalty that choice costs.

Answer Instructions

- Pick any two relevant design decisions.
- Write roughly 2–3 sentences per decision.
- Don't answer using only "HPK uses LVT, PMK uses HVT" — that part of the story is already given.
- Frame your reasoning around how performance, leakage power, area, capacitance, and cell architecture pull against each other.

Areas to Consider

You may consider factors such as:

- Transistor-sizing philosophy
- Logic topology or transistor stack depth
- Available drive-strength range
- Transistor channel length
- PMOS-to-NMOS sizing ratio